KUBERNETES USER MANAGEMENT

Step-by-step assignment guide

Student ID: 02230130
Name: ___Kelzang Penjor_____
Platform: Docker Desktop on Windows with minikube

Includes milestone instructions, commands, application code, and Kubernetes manifests

How to use this guide

Follow the milestones in order. Commands are written for Windows PowerShell unless a section is explicitly labeled Git Bash. Run commands from the project folder D:\ITM403\Assignment. Capture the outputs requested in the evidence lists as you go. The Secret YAML is generated locally and must not be committed to a public repository.

The assignment document is the source of the marking requirements. This guide supplies one working implementation for the required API and Kubernetes resources; replace only the identity fields if your instructor gives different instructions. The real-name field is intentionally blank.

Contents

- 1. Prerequisites and platform setup
- 2. Milestone 1: cluster and namespace
- 3. Milestone 2: PostgreSQL and persistence
- 4. Milestone 3: REST API and CRUD test
- 5. Milestone 4: Ingress and hostname routing
- 6. Milestone 5: HPA and VPA
- 7. Milestone 6: report, evidence, and demo answers
- Appendix A: all project source and manifests

1. The Scenario

A small organisation needs an internal user management service: an API that creates, lists, updates, and deletes user records. Because staff will use it every day, it must keep its data when Pods restart, be reachable at a friendly URL, and cope with busy periods without manual intervention.

Your task is to build and deploy this service on a local Kubernetes cluster. Figure 1 shows the target architecture. You will build it in six milestones; each milestone assesses a separate group of Kubernetes concepts and is marked independently, so a problem in one milestone does not prevent you earning marks in the others.

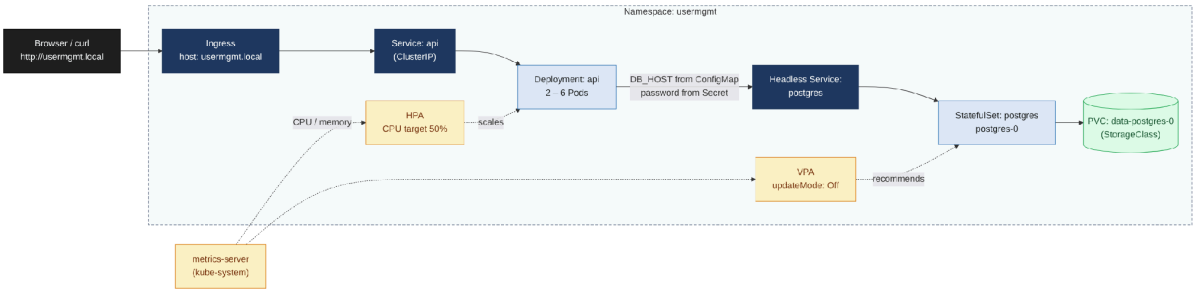


Figure 1 — Target architecture for the user management service.

.

Before the milestones — required tools

Start Docker Desktop. If Git is already installed, install minikube and kubectl from Administrator PowerShell:

```
winget install -e --id Kubernetes.minikube
winget install -e --id Kubernetes.kubectl
```

Close and reopen PowerShell, then check the tools:

```
docker version
git --version
minikube version
kubectl version --client
```

Milestone 1 — Cluster and platform setup

Task: Start minikube with at least 2 CPUs and 4 GB memory

Use: Docker Desktop must be running with Linux containers.

```
minikube start --driver=docker --cpus=2 --memory=4096
kubectl get nodes
```

Task: Enable Ingress and metrics-server

Use: Traefik, matching `ingressClassName: traefik` in `k8s/06-ingress.yaml`. This add-on requires minikube 1.39.0 or later.

```
minikube addons enable metrics-server
minikube addons enable traefik
kubectl get ingressclass
kubectl get pods -A
kubectl top nodes
```

Task: Install the Vertical Pod Autoscaler components

Use: Git Bash, from a folder outside this repository. The script installs the cluster platform component.

```
git clone https://github.com/kubernetes/autoscaler.git
cd autoscaler/vertical-pod-autoscaler
./hack/vpa-up.sh
kubectl get pods -n kube-system | grep vpa
```

Wait until the recommender, updater, and admission-controller Pods show `Running`:

```
kubectl get pods -A
```

Task: Create the SID-specific namespace from a manifest

File: `k8s/01-namespace.yaml` defines `usermgmt-02230130`.

```
kubectl apply -f .\k8s\01-namespace.yaml
kubectl get namespace usermgmt-02230130
```

Task: Identify the default StorageClass and provisioner

```
kubectl get storageclass
kubectl get storageclass standard -o yaml
```

The default class is marked `(default)` in the first command. Read its `provisioner` field in the YAML output. If the class has a different name, substitute that name in the second command.

Evidence for Milestone 1

`kubectl get nodes`,

```
PS D:\ITM403\Assignment> kubectl get nodes
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   17h   v1.37.0
PS D:\ITM403\Assignment>
```

`kubectl get pods -A`,

```
PS D:\ITM403\Assignment> kubectl get pods -A
NAMESPACE     NAME                               READY   STATUS    RESTARTS   AGE
kube-system   coredns-559f6c778d-6bw52           1/1     Running   0          17h
kube-system   coredns-559f6c778d-j7j59           1/1     Running   0          17h
kube-system   etcd-minikube                      1/1     Running   0          17h
kube-system   kindnet-4mrg9                      1/1     Running   0          17h
kube-system   kube-apiserver-minikube            1/1     Running   0          17h
kube-system   kube-controller-manager-minikube   1/1     Running   0          17h
kube-system   kube-proxy-psqsk                   1/1     Running   0          17h
kube-system   kube-scheduler-minikube            1/1     Running   0          17h
kube-system   metrics-server-768f9f6999-jhl69     1/1     Running   0          115m
kube-system   storage-provisioner                1/1     Running   0          17h
kube-system   traefik-776cf48b84-bbp2p           1/1     Running   0          113m
PS D:\ITM403\Assignment>
```

`kubectl get storageclass`, and

```
PS D:\ITM403\Assignment> kubectl get storageclass
NAME                 PROVISIONER                RECLAIMPOLICY   VOLUMEBINDINGMODE   ALLOWVOLUMEEXPANSION   AGE
standard (default)   k8s.io/minikube-hostpath   Delete          Immediate           false                  17h
PS D:\ITM403\Assignment>
```

`kubectl top nodes`.

```
PS D:\ITM403\Assignment> kubectl top nodes
NAME       CPU(cores)   CPU(%)   MEMORY(bytes)   MEMORY(%)
minikube   674m         5%       945Mi           26%
PS D:\ITM403\Assignment>
```

Milestone 2 — Stateful PostgreSQL database

Task: Create a Secret for database name, username, and password

File: `scripts/New-DbSecret.ps1` asks for a password and generates `k8s/02-db-secret.yaml`. The generated Secret is ignored by Git. Database and username are both `usermgmt`.

```
.\scripts\New-DbSecret.ps1
kubectl apply -f .\k8s\02-db-secret.yaml
kubectl get secret postgres-secret -n usermgmt-02230130
```

Do not commit the generated Secret. Base64 values in a Kubernetes Secret are not encryption.

Task: Create a headless Service named postgres

File: `k8s/03-postgres.yaml` contains the Service with `clusterIP: None` and port `5432`.

```
kubectl apply -f .\k8s\03-postgres.yaml
kubectl get service postgres -n usermgmt-02230130
```

Task: Create a one-replica PostgreSQL StatefulSet using Secret credentials

File: `k8s/03-postgres.yaml` defines the `postgres` StatefulSet and reads all three PostgreSQL values from `postgres-secret`.

```
kubectl rollout status statefulset/postgres -n usermgmt-02230130 --timeout=180s
kubectl get statefulset,pod -n usermgmt-02230130
```

Expected Pod name: `postgres-0`.

Task: Give the StatefulSet a per-Pod PVC of at least 1 Gi

File: In `k8s/03-postgres.yaml`, `volumeClaimTemplates` requests `1Gi` with `ReadWriteOnce`.

```
kubectl get pvc,pv -n usermgmt-02230130
```

Task: Set database resource requests and limits

File: In the same StatefulSet container, `resources.requests` are `100m` CPU and `256Mi` memory; limits are `500m` CPU and `512Mi` memory.

```
kubectl describe pod postgres-0 -n usermgmt-02230130
```

Task: Prove that data survives deletion of postgres-0

Run this in PowerShell to create and read one row:

```
@'
CREATE TABLE IF NOT EXISTS persistence_check (id SERIAL PRIMARY KEY, note TEXT NOT NULL);
INSERT INTO persistence_check (note) VALUES ('sid-02230130-persisted');
SELECT * FROM persistence_check ORDER BY id DESC LIMIT 1;
'@ | kubectl exec -i -n usermgmt-02230130 postgres-0 -- psql -U usermgmt -d usermgmt
```

Delete the Pod, wait for its replacement, and query the same row:

```
kubectl delete pod postgres-0 -n usermgmt-02230130
kubectl wait --for=condition=Ready pod/postgres-0 -n usermgmt-02230130 --timeout=180s
@'
SELECT * FROM persistence_check ORDER BY id DESC LIMIT 1;
'@ | kubectl exec -i -n usermgmt-02230130 postgres-0 -- psql -U usermgmt -d usermgmt
kubectl get statefulset,pod,pvc,pv -n usermgmt-02230130
```

The row should still appear after the Pod is recreated. Save output from before deletion and after recreation.

Evidences For Milestone 2

`kubectl get statefulset,pod,pvc,pv -n usermgmt-<sid>`; and the persistence test as a sequence of commands and outputs (before deletion, deletion, after recreation).

```
PS D:\ITM403\Assignment> kubectl get statefulset,pod,pvc,pv -n usermgmt-02230130
NAME                          READY   AGE
statefulset.apps/postgres     1/1     10m

NAME             READY   STATUS    RESTARTS   AGE
pod/postgres-0   1/1     Running   0          2m3s

NAME                                          STATUS   VOLUME
                CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
persistentvolumeclaim/postgres-data-postgres-0   Bound    pvc-7a32ca49-7635-4c91-9a7d-
b697ea53305d   1Gi        RWO            standard       <unset>                 9m59s

NAME                                                        CAPACITY   ACCESS MODES
RECLAIM POLICY   STATUS   CLAIM                                       STORAGECLASS
VOLUMEATTRIBUTESCLASS   REASON   AGE
persistentvolume/pvc-7a32ca49-7635-4c91-9a7d-b697ea53305d   1Gi        RWO
Delete           Bound    usermgmt-02230130/postgres-data-postgres-0   standard
<unset>                           9m59s
PS D:\ITM403\Assignment>
```

And the persistence test as a sequence of commands and outputs:

Before deletion:

```
PS D:\ITM403\Assignment> @'
>> CREATE TABLE IF NOT EXISTS persistence_check (id SERIAL PRIMARY KEY, note TEXT NOT
NULL);
>> INSERT INTO persistence_check (note) VALUES ('sid-02230130-persisted');
>> SELECT * FROM persistence_check ORDER BY id DESC LIMIT 1;
>> '@ | kubectl exec -i -n usermgmt-02230130 postgres-0 -- psql -U usermgmt -d usermgm
t
NOTICE:  relation "persistence_check" already exists, skipping
CREATE TABLE
INSERT 0 1
 id |         note
----+----------------------
  2 | sid-02230130-persisted
(1 row)

PS D:\ITM403\Assignment>
```

Deletion:

```
PS D:\ITM403\Assignment> kubectl delete pod postgres-0 -n usermgmt-02230130
pod "postgres-0" deleted from usermgmt-02230130 namespace
PS D:\ITM403\Assignment>
```

After recreation:

```
PS D:\ITM403\Assignment> @'
>> SELECT * FROM persistence_check ORDER BY id DESC LIMIT 1;
>> '@ | kubectl exec -i -n usermgmt-02230130 postgres-0 -- psql -U usermgmt -d usermgm
t
 id |          note
----+------------------------
  2 | sid-02230130-persisted
(1 row)

PS D:\ITM403\Assignment>
```

Milestone 3 — User management API

Task: Write the REST API and store user records in PostgreSQL

File: `app/main.py` implements `GET /users`, `POST /users`, `GET /users/{id}`, `PUT /users/{id}`, `DELETE /users/{id}`, `GET /healthz`, and `GET /ready`. The ready endpoint checks the database. Database settings come from environment variables; the password is not in the source code.

Task: Write a Dockerfile, build a versioned image, and load it into minikube

Files: `app/Dockerfile`, `app/requirements.txt`, and `app/main.py`.

```
docker build -t usermgmt-api:1.0 .\app
minikube image load usermgmt-api:1.0
```

Keep the tag `1.0`; do not use `latest`.

Task: Create a ConfigMap for non-secret database settings

File: `k8s/04-api-config.yaml` sets `DB_HOST=postgres`, port `5432`, database name, and username.

```
kubectl apply -f .\k8s\04-api-config.yaml
kubectl get configmap api-config -n usermgmt-02230130 -o yaml
```

Task: Deploy two API replicas, reading config from ConfigMap and password from Secret

File: `k8s/05-api.yaml` references `api-config` and the `DB_PASSWORD` key in `postgres-secret`.

```
kubectl apply -f .\k8s\05-api.yaml
kubectl rollout status deployment/api -n usermgmt-02230130 --timeout=180s
kubectl get deploy,rs,pods -n usermgmt-02230130
```

Task: Add liveness and readiness probes

File: `k8s/05-api.yaml` uses `/healthz` for liveness and `/ready` for readiness. `/ready` returns success only when PostgreSQL responds.

```
kubectl describe deployment api -n usermgmt-02230130
```

Task: Set API requests and limits

File: `k8s/05-api.yaml` requests `100m` CPU and `128Mi` memory; limits are `500m` CPU and `256Mi` memory. The CPU request is needed for HPA utilization calculations.

Task: Create a ClusterIP Service named api

File: `k8s/05-api.yaml` also creates the `api` Service on port `80` targeting container port `8080`.

```
kubectl get service api -n usermgmt-02230130
```

Task: Demonstrate all CRUD endpoints through port-forward

In a second PowerShell window, start the port-forward and leave it running:

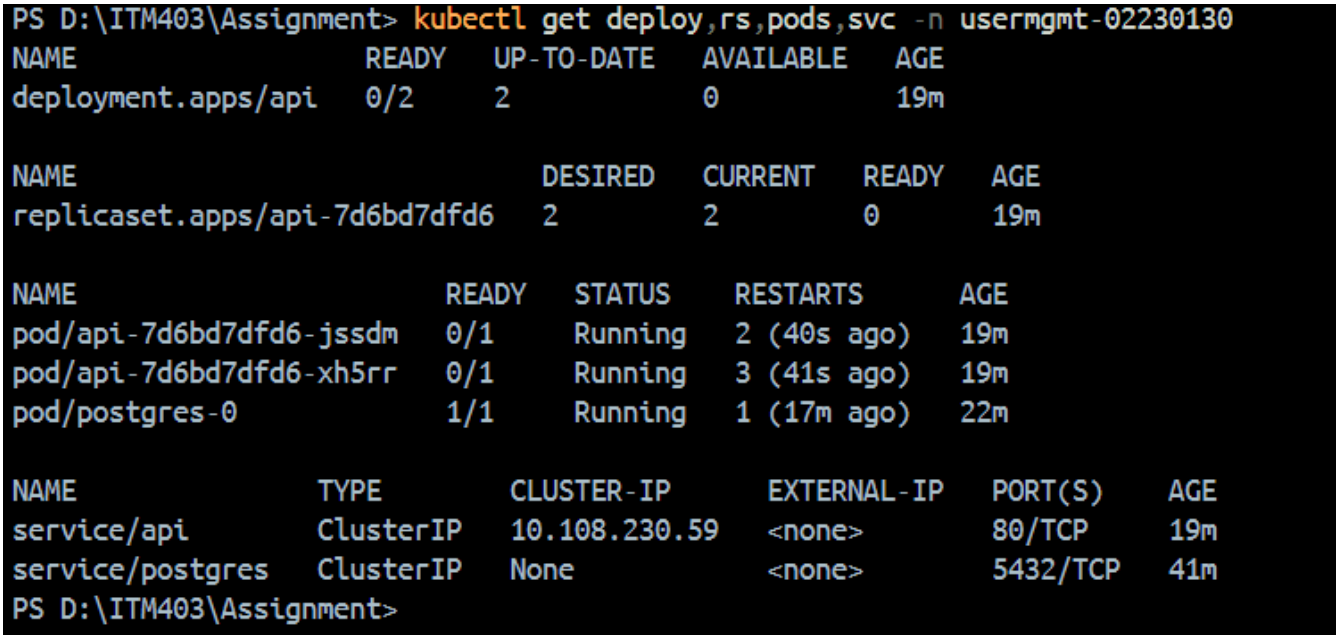
```
kubectl port-forward -n usermgmt-02230130 svc/api 8080:80
```

In the first window, run each endpoint. The created ID is stored in `$created.id` and reused in the later calls.

```
$base = 'http://127.0.0.1:8080'
$created = Invoke-RestMethod -Method Post -Uri "$base/users" -ContentType 'application/json' -Body '{"name":"SID
02230130 User","email":"sid02230130@example.test"}'
$created
Invoke-RestMethod -Method Get -Uri "$base/users"
Invoke-RestMethod -Method Get -Uri "$base/users/$($created.id)"
Invoke-RestMethod -Method Put -Uri "$base/users/$($created.id)" -ContentType 'application/json' -Body
'{"name":"Updated SID 02230130 User","email":"updated02230130@example.test"}'
Invoke-RestMethod -Method Delete -Uri "$base/users/$($created.id)"
```

Evidence for Milestone 3

```
kubectl get deploy,rs,pods,svc -n usermgmt-02230130
```



Capture the Dockerfile, manifests, and successful response for every CRUD call.

CRUD test through `kubectl port-forward svc/api 8080:80` showing each endpoint working.

```
PS D:\ITM403\Assignment> kubectl port-forward -n usermgmt-02230130 svc/api 8080:80
Forwarding from 127.0.0.1:8080 -> 8080
Forwarding from [::1]:8080 -> 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
```

CREATE — Add a user

```
PS D:\ITM403\Assignment> $created = Invoke-RestMethod -Method Post -Uri "$base/users"
-ContentType 'application/json' -Body '{"name":"SID 02230130 User","email":"sid0223013
0@example.test"}'
>> $created

email                    id name
-----                    -- ----
sid02230130@example.test  2 SID 02230130 User


PS D:\ITM403\Assignment>
```

READ — Get all users

```
PS D:\ITM403\Assignment> Invoke-RestMethod -Method Get -Uri "$base/users"

email                    id name
-----                    -- ----
sid02230130@example.test  2 SID 02230130 User


PS D:\ITM403\Assignment>
```

READ — Get the specific user

```
PS D:\ITM403\Assignment> Invoke-RestMethod -Method Get -Uri "$base/users/$($created.id
)"

email                    id name
-----                    -- ----
sid02230130@example.test  2 SID 02230130 User


PS D:\ITM403\Assignment>
```

UPDATE — Modify the user:

```
PS D:\ITM403\Assignment> Invoke-RestMethod -Method Put -Uri "$base/users/$($created.id
)" -ContentType 'application/json' -Body '{"name":"Updated SID 02230130 User","email":
"updated02230130@example.test"}'

email                        id name
-----                        -- ----
updated02230130@example.test  2 Updated SID 02230130 User


PS D:\ITM403\Assignment>
```

DELETE — Remove the user

```
PS D:\ITM403\Assignment> Invoke-RestMethod -Method Delete -Uri "$base/users/$($created
.id)"

email                        id name
-----                        -- ----
updated02230130@example.test  2 Updated SID 02230130 User


PS D:\ITM403\Assignment>
```

Verify deletion:

```
PS D:\ITM403\Assignment> Invoke-RestMethod -Method Get -Uri "$base/users" | ConvertTo-
Json -Depth 5
{
    "value":  [

               ],
    "Count":  0
}
PS D:\ITM403\Assignment>
```

Milestone 4 — Ingress and host routing

Task: Route usermgmt-02230130.local to Service api

File: `k8s/06-ingress.yaml` sends the `/` path to Service `api` on port `80`.

Task: Set ingressClassName to match the installed controller

The setup above uses Traefik. Check its class and apply the manifest:

```
kubectl get ingressclass
kubectl apply -f .\k8s\06-ingress.yaml
kubectl describe ingress -n usermgmt-02230130
```

Task: Add the host name to Windows hosts file

Start the tunnel in a separate elevated PowerShell window and leave it running:

```
minikube tunnel
```

Open `C:\Windows\System32\drivers\etc\hosts` as Administrator and add:

```
127.0.0.1 usermgmt-02230130.local
```

Task: Show that the correct host works and a different host does not

```
curl.exe -i http://usermgmt-02230130.local/users
curl.exe -i -H "Host: other.local" http://127.0.0.1/users
```

The first request should reach the API.

```
PS D:\ITM403\Assignment> curl.exe -i http://usermgmt-02230130.local/users
HTTP/1.1 200 OK
Content-Length: 3
Content-Type: application/json
Date: Mon, 19 Oct 2026 15:38:38 GMT
Server: gunicorn

[]
PS D:\ITM403\Assignment>
```

The second should return the Ingress controller's default 404. Save both results and the hosts file line.

```
PS D:\ITM403\Assignment> curl.exe -i -H "Host: other.local" http://127.0.0.1/users
HTTP/1.1 404 Not Found
Content-Type: text/plain; charset=utf-8
X-Content-Type-Options: nosniff
Date: Mon, 19 Oct 2026 15:43:17 GMT
Content-Length: 19

404 page not found
PS D:\ITM403\Assignment>
```

Evidence for Milestone 4

Capture `kubectl describe ingress -n usermgmt-02230130`,

```
PS D:\ITM403\Assignment> kubectl describe ingress -n usermgmt-02230130
Name:             api
Labels:           <none>
Namespace:        usermgmt-02230130
Address:          127.0.0.1
Ingress Class:    traefik
Default backend:  <default>
Rules:
  Host                   Path  Backends
  ----                   ----  --------
  usermgmt-02230130.local
                         /   api:80 (10.244.0.9:8080,10.244.0.10:8080)
Annotations:             <none>
Events:                  <none>
PS D:\ITM403\Assignment>
```

The hosts-file entry, the successful expected-host response, and the other-host failure.

Milestone 5 — Metrics, HPA, and VPA

Task: Confirm pod metrics are available

```
kubectl top pods -n usermgmt-02230130
```

Task: Create an HPA for api with 2–6 replicas and 50% CPU target

File: `k8s/07-hpa.yaml` defines `autoscaling/v2`, min replicas `2`, max replicas `6`, and average CPU utilization `50%`.

```
kubectl apply -f .\k8s\07-hpa.yaml
kubectl get hpa -n usermgmt-02230130
```

Task: Generate load and record replica count during and after the load

In one terminal, run the assignment load command for three minutes:

```
hey -z 3m -c 50 http://usermgmt-02230130.local/users
```

In another PowerShell window, start a transcript and record HPA and Deployment state every 30 seconds. Leave the loop running for several minutes after the load finishes, then press Ctrl+C and run `Stop-Transcript`.

```
Start-Transcript -Path .\hpa-timeline.txt -Append
while ($true) {
  Get-Date -Format 'yyyy-MM-dd HH:mm:ss'
  kubectl get hpa api -n usermgmt-02230130
  kubectl get deployment api -n usermgmt-02230130
  Start-Sleep -Seconds 30
}
Stop-Transcript
```

Use the transcript output to make a report timeline showing time, HPA CPU, and desired/current replicas during load and scale-down.

Task: Create VPA for postgres with updateMode Off and record its recommendations

File: `k8s/08-vpa.yaml` targets the PostgreSQL StatefulSet with `updateMode: "Off"`.

```
kubectl apply -f .\k8s\08-vpa.yaml
kubectl describe vpa postgres -n usermgmt-02230130
```

Capture the VPA target, lower bound, and upper bound from the output after the load run.

Report question: Why put VPA on postgres and use Off mode?

Use this answer in your own words: HPA changes the number of API Pods based on CPU utilization. VPA can change CPU requests, which changes the utilization percentage HPA observes if both act on the same Deployment. This setup puts VPA on PostgreSQL and leaves it in `Off` mode, so it reports resource recommendations without changing or restarting the database Pod during the demonstration.

Evidence for Milestone 5

Capture `kubectl top pods`,

```
PS D:\ITM403\Assignment> kubectl top pod postgres-0 -n usermgmt-02230130
NAME         CPU(cores)   MEMORY(bytes)
postgres-0   59m          39Mi
```

The HPA/VPA manifests, `hpa-timeline.txt`,

```
api    Deployment/api   cpu: 7%/50%   2         6         2         93m
NAME   READY   UP-TO-DATE   AVAILABLE   AGE
api    2/2     2            2           142m
2026-10-19 23:22:36
NAME   REFERENCE        TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
api    Deployment/api   cpu: 6%/50%   2         6         2          94m
NAME   READY   UP-TO-DATE   AVAILABLE   AGE
api    2/2     2            2           142m
2026-10-19 23:23:08
NAME   REFERENCE        TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
api    Deployment/api   cpu: 6%/50%   2         6         2          94m
NAME   READY   UP-TO-DATE   AVAILABLE   AGE
api    2/2     2            2           143m
2026-10-19 23:23:38
NAME   REFERENCE        TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
api    Deployment/api   cpu: 5%/50%   2         6         2          95m
NAME   READY   UP-TO-DATE   AVAILABLE   AGE
api    2/2     2            2           143m
2026-10-19 23:24:09
NAME   REFERENCE        TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
api    Deployment/api   cpu: 5%/50%   2         6         2          96m
NAME   READY   UP-TO-DATE   AVAILABLE   AGE
```

And `kubectl describe vpa postgres -n usermgmt-02230130`.

```
PS D:\ITM403\Assignment>  kubectl describe vpa postgres -n usermgmt-02230130
Name:         postgres
Namespace:    usermgmt-02230130
Labels:       <none>
Annotations:  <none>
API Version:  autoscaling.k8s.io/v1
Kind:         VerticalPodAutoscaler
Metadata:
  Creation Timestamp:  2026-10-19T17:25:36Z
  Generation:          1
  Resource Version:    68936
  UID:                 44498850-be65-45f6-9328-798c9fb2a77b
Spec:
  Target Ref:
    API Version:  apps/v1
    Kind:         StatefulSet
    Name:         postgres
  Update Policy:
    Update Mode:  Off
Events:           <none>
PS D:\ITM403\Assignment>
```

Appendix A: complete project code

The following listings are the exact source and manifest files used by the walkthrough. Create or replace the matching files in the project tree. The generated Secret file is excluded because its password is personal to your local cluster.

app/main.py

Python Flask API source

```
import os
import time

import psycopg
from flask import Flask, jsonify, request
from psycopg.rows import dict_row

app = Flask(__name__)

def connect():
    return psycopg.connect(host=os.environ.get("DB_HOST", "postgres"), port=os.environ.get("DB_PORT", "5432"),
dbname=os.environ["DB_NAME"], user=os.environ["DB_USER"], password=os.environ["DB_PASSWORD"], connect_timeout=2,
row_factory=dict_row)

def initialize_database():
    for attempt in range(30):
        try:
            with connect() as conn, conn.cursor() as cur:
                cur.execute("CREATE TABLE IF NOT EXISTS users (id SERIAL PRIMARY KEY, name TEXT NOT NULL, email TEXT
NOT NULL UNIQUE)")
            return
        except psycopg.OperationalError:
            if attempt == 29:
                raise
            time.sleep(2)

@app.get("/healthz")
def healthz():
    return {"status": "ok"}, 200

@app.get("/ready")
def ready():
    try:
        with connect() as conn:
            conn.execute("SELECT 1")
        return {"status": "ready"}, 200
    except psycopg.Error:
        return {"status": "not ready"}, 503

@app.get("/users")
def list_users():
    with connect() as conn:
        rows = conn.execute("SELECT id, name, email FROM users ORDER BY id").fetchall()
    return jsonify(rows)

@app.post("/users")
def create_user():
    data = request.get_json(silent=True) or {}
    if not data.get("name") or not data.get("email"):
        return {"error": "name and email are required"}, 400
    try:
        with connect() as conn:
            row = conn.execute("INSERT INTO users (name, email) VALUES (%s, %s) RETURNING id, name, email",
(data["name"], data["email"])).fetchone()
        return jsonify(row), 201
    except psycopg.errors.UniqueViolation:
        return {"error": "email already exists"}, 409
```

```
@app.get("/users/<int:user_id>")
def get_user(user_id):
    with connect() as conn:
        row = conn.execute("SELECT id, name, email FROM users WHERE id = %s", (user_id,)).fetchone()
    return (jsonify(row), 200) if row else ({"error": "user not found"}, 404)

@app.put("/users/<int:user_id>")
def update_user(user_id):
    data = request.get_json(silent=True) or {}
    if not data.get("name") or not data.get("email"):
        return {"error": "name and email are required"}, 400
    try:
        with connect() as conn:
            row = conn.execute("UPDATE users SET name = %s, email = %s WHERE id = %s RETURNING id, name, email",
(data["name"], data["email"], user_id)).fetchone()
        return (jsonify(row), 200) if row else ({"error": "user not found"}, 404)
    except psycopg.errors.UniqueViolation:
        return {"error": "email already exists"}, 409

@app.delete("/users/<int:user_id>")
def delete_user(user_id):
    with connect() as conn:
        row = conn.execute("DELETE FROM users WHERE id = %s RETURNING id, name, email", (user_id,)).fetchone()
    return (jsonify(row), 200) if row else ({"error": "user not found"}, 404)

if __name__ == "__main__":
    initialize_database()
    app.run(host="0.0.0.0", port=8080)
```

app/requirements.txt

Python package versions

```
Flask==3.1.1
psycopg[binary]==3.2.9
gunicorn==23.0.0
```

app/Dockerfile

Container build instructions

```
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY main.py .
EXPOSE 8080
CMD ["sh", "-c", "python -c 'from main import initialize_database; initialize_database()' && gunicorn --bind
0.0.0.0:8080 --workers 2 --threads 4 main:app"]
```

scripts/New-DbSecret.ps1

PowerShell helper that generates the local Secret manifest

```
$ErrorActionPreference = 'Stop'
$passwordSecure = Read-Host 'Enter a database password' -AsSecureString
$password = [System.Net.NetworkCredential]::new('', $passwordSecure).Password
if ([string]::IsNullOrWhiteSpace($password)) { throw 'Password cannot be empty.' }
$encoded = [Convert]::ToBase64String([Text.Encoding]::UTF8.GetBytes($password))
$content = @"
apiVersion: v1
kind: Secret
metadata:
  name: postgres-secret
  namespace: usermgmt-02230130
type: Opaque
data:
```

```
  POSTGRES_DB: dXNlcm1nbXQ=
  POSTGRES_USER: dXNlcm1nbXQ=
  POSTGRES_PASSWORD: $encoded
  DB_NAME: dXNlcm1nbXQ=
  DB_USER: dXNlcm1nbXQ=
  DB_PASSWORD: $encoded
"@
$path = Join-Path $PSScriptRoot '..\k8s\02-db-secret.yaml'
Set-Content -Path $path -Value $content -Encoding ascii
Write-Host "Wrote Secret manifest to $((Resolve-Path $path).Path). It is git-ignored."
```

k8s/01-namespace.yaml

Namespace

```
apiVersion: v1
kind: Namespace
metadata:
  name: usermgmt-02230130
```

k8s/03-postgres.yaml

Headless Service and PostgreSQL StatefulSet

```
apiVersion: v1
kind: Service
metadata:
  name: postgres
  namespace: usermgmt-02230130
spec:
  clusterIP: None
  selector:
    app: postgres
  ports:
    - name: postgres
      port: 5432
      targetPort: 5432
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
  namespace: usermgmt-02230130
spec:
  serviceName: postgres
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:16
          ports:
            - name: postgres
              containerPort: 5432
          env:
            - name: POSTGRES_DB
              valueFrom:
                secretKeyRef:
                  name: postgres-secret
                  key: POSTGRES_DB
            - name: POSTGRES_USER
              valueFrom:
                secretKeyRef:
                  name: postgres-secret
                  key: POSTGRES_USER
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: postgres-secret
                  key: POSTGRES_PASSWORD
          resources:
            requests:
              cpu: 100m
              memory: 256Mi
            limits:
              cpu: 500m
              memory: 512Mi
          readinessProbe:
            exec:
              command: ["sh", "-c", "pg_isready -U \"$POSTGRES_USER\" -d \"$POSTGRES_DB\""]
            initialDelaySeconds: 5
            periodSeconds: 5
          livenessProbe:
```

```
            exec:
              command: ["sh", "-c", "pg_isready -U \"$POSTGRES_USER\" -d \"$POSTGRES_DB\""]
            initialDelaySeconds: 20
            periodSeconds: 10
          volumeMounts:
            - name: postgres-data
              mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:
    - metadata:
        name: postgres-data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 1Gi
```

k8s/04-api-config.yaml

Non-secret API configuration

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: api-config
  namespace: usermgmt-02230130
data:
  DB_HOST: postgres
  DB_PORT: "5432"
  DB_NAME: usermgmt
  DB_USER: usermgmt
```

k8s/05-api.yaml

API Deployment and ClusterIP Service

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
  namespace: usermgmt-02230130
spec:
  replicas: 2
  selector:
    matchLabels:
      app: api
  template:
    metadata:
      labels:
        app: api
    spec:
      containers:
        - name: api
          image: usermgmt-api:1.0
          imagePullPolicy: IfNotPresent
          ports:
            - name: http
              containerPort: 8080
          envFrom:
            - configMapRef:
                name: api-config
          env:
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: postgres-secret
                  key: DB_PASSWORD
          resources:
            requests:
              cpu: 100m
              memory: 128Mi
            limits:
              cpu: 500m
              memory: 256Mi
          livenessProbe:
            httpGet:
              path: /healthz
              port: http
            initialDelaySeconds: 10
            periodSeconds: 10
          readinessProbe:
            httpGet:
              path: /ready
              port: http
            initialDelaySeconds: 5
            periodSeconds: 5
---
apiVersion: v1
kind: Service
metadata:
  name: api
  namespace: usermgmt-02230130
spec:
  type: ClusterIP
  selector:
    app: api
  ports:
    - name: http
      port: 80
      targetPort: http
```

k8s/06-ingress.yaml

Host-based Ingress

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: api
  namespace: usermgmt-02230130
spec:
  ingressClassName: traefik


  rules:
    - host: usermgmt-02230130.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: api
                port:
                  number: 80
```

k8s/07-hpa.yaml

HorizontalPodAutoscaler

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api
  namespace: usermgmt-02230130
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api
  minReplicas: 2
  maxReplicas: 6
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50
```

k8s/08-vpa.yaml

VerticalPodAutoscaler

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: postgres
  namespace: usermgmt-02230130
spec:
  targetRef:
    apiVersion: apps/v1
    kind: StatefulSet
    name: postgres
  updatePolicy:
    updateMode: "Off"
```